УО «Белорусский государственный университет информатики и радиоэлектроники»

Кафедра ПОИТ

Отчет по лабораторной работе №6

по предмету «Методы машинного обучения»

Выполнил:
Бражалович А.И.
Гр. 351004
Проверил:
Воронина А. М.

Минск 2026

# 1 ЦЕЛЬ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Целью выполнения лабораторной работы является изучение и практическое освоение ансамблевых методов машинного обучения, а также приобретение навыков построения и настройки модели градиентного бустинга. В работе требуется подготовить и предобработать набор данных, обучить модель градиентного бустинга с исходными параметрами, оценить её качество по метрикам классификации, визуализировать результаты (матрицу ошибок и ROC-кривую), выполнить подбор гиперпараметров с помощью GridSearchCV, повторить обучение и оценку с оптимальными параметрами, а также исследовать влияние скорости обучения (learning_rate) и количества деревьев на качество модели.

# 2 ТЕОРИЯ ПО ЛАБОРАТОРНОЙ РАБОТЕ

## 2.1 Ансамблевые методы

Ансамблевые методы объединяют несколько базовых алгоритмов машинного обучения в одну композицию для получения более мощной и устойчивой модели. Основной недостаток одиночного дерева решений — склонность к переобучению и высокая чувствительность к изменению обучающей выборки. Объединение алгоритмов в ансамбли позволяет снизить ошибку обобщения. Ошибка на новых данных раскладывается на шум, смещение (bias) и разброс (variance). Бэггинг уменьшает разброс за счёт усреднения независимых моделей, а бустинг уменьшает смещение за счёт последовательного исправления ошибок предыдущих моделей.

## 2.2 Бэггинг и случайный лес

Бэггинг (bootstrap aggregating) формирует N бутстрап-подвыборок из исходных данных (случайный выбор с возвращением), на каждой из которых независимо обучается базовый алгоритм. Итоговый ответ получается усреднением (регрессия) или голосованием (классификация). Случайный лес дополнительно использует метод случайных подпространств: в каждом узле признак выбирается из случайного подмножества размера m (рекомендуется m = sqrt(d) для классификации). Это снижает корреляцию деревьев и уменьшает дисперсию ансамбля.

## 2.3 Градиентный бустинг

Бустинг — последовательное построение ансамбля, в котором каждое следующее дерево обучается исправлять ошибки уже построенной композиции. В градиентном бустинге на каждой итерации вычисляется вектор сдвига (антиградиент функции потерь), на нём обучается новое базовое дерево, после чего оно добавляется в ансамбль с масштабированием на скорость обучения: a_n(x) = a_{n-1}(x) + eta * b_n(x), где eta принадлежит (0, 1]. В задачах

классификации в качестве функции потерь используется логарифмическая (log loss).

### 2.4 Скорость обучения и переобучение

Скорость обучения eta определяет вклад каждого нового дерева. Малое значение eta приводит к медленной сходимости, но обычно даёт лучшее обобщение и требует большего числа деревьев. Большое значение eta ускоряет обучение, однако повышает риск переобучения. Градиентный бустинг склонен к переобучению при увеличении числа итераций N или глубины деревьев, поэтому оптимальные параметры подбираются по отложенной выборке или с помощью кросс-валидации (например, GridSearchCV).

### 2.5 Метрики качества классификации

Accuracy — доля правильных ответов. Precision — доля истинно положительных среди предсказанных положительных. Recall — доля правильно найденных положительных объектов. F1-score — гармоническое среднее precision и recall. ROC AUC — площадь под ROC-кривой, показывающая качество ранжирования объектов по вероятности положительного класса и устойчивая к дисбалансу классов.

## 3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Для нечётного варианта (вариант 3) индивидуальное задание посвящено градиентному бустингу и включает следующие этапы:

1) подготовить и проанализировать набор данных, выполнить очистку и предобработку (обработка пропусков, стандартизация, кодирование категориальных признаков);

2) обучить модель градиентного бустинга с исходными параметрами на тренировочной части данных и сделать прогнозы на тестовой части;

3) оценить качество модели по метрикам accuracy, precision, recall, F1-score, ROC AUC;

4) визуализировать результаты (матрица ошибок, ROC-кривая);

5) провести настройку гиперпараметров модели (число деревьев, глубина деревьев, скорость обучения) с использованием GridSearchCV;

6) повторить обучение и оценку с новыми параметрами и сравнить результаты;

7) исследовать влияние скорости обучения (learning_rate) и количества деревьев на качество модели.

Все этапы выполнения оформлены в виде оконного приложения на языке Python (библиотека Tkinter) с вкладками «Данные и предобработка», «Обучение и оценка», «Подбор гиперпараметров» и «Влияние параметров».

# 4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

## 4.1 Модуль lab6.py

```
import os
import warnings
warnings.filterwarnings('ignore')


import tkinter as tk
from tkinter import ttk


import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import seaborn as sns


from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve,
                             confusion_matrix, classification_report)


SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))


CSV_CANDIDATES = [
    os.path.join(SCRIPT_DIR, 'Bankloan.csv'),
    os.path.join(SCRIPT_DIR, '..', '..', 'lab4', 'code', 'Bankloan.csv'),
    os.path.join(SCRIPT_DIR, '..', '..', 'lab5', 'Bankloan.csv'),
]


TARGET = 'default'
CONTINUOUS_COLS = ['age', 'employ', 'address', 'income', 'debtinc',
                   'creddebt', 'othdebt']
METRIC_NAMES = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
METRIC_RUS = {
    'accuracy': 'Accuracy',
    'precision': 'Precision',
    'recall': 'Recall',
    'f1': 'F1',
    'roc_auc': 'ROC AUC',
}



def load_raw():
    for path in CSV_CANDIDATES:
        if os.path.exists(path):
            return pd.read_csv(path)
    return None



def clean_data(df):
    df = df.copy()
    df = df.dropna(subset=[TARGET])
    for col in CONTINUOUS_COLS:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        df[col] = df[col].clip(lower=q1 - 1.5 * iqr, upper=q3 + 1.5 * iqr)
    df[TARGET] = df[TARGET].astype(int)
    return df
```

```
def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]
    metrics = {
        'accuracy': accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred),
        'roc_auc': roc_auc_score(y_test, y_proba),
    }
    return metrics, y_pred, y_proba



def fig_to_tab(fig, parent):
    canvas = FigureCanvasTkAgg(fig, master=parent)
    canvas.get_tk_widget().pack(fill='both', expand=True)
    return canvas



class Lab6App:
    def __init__(self, root):
        self.root = root
        root.title("ЛР 06. Ансамбли: градиентный бустинг")
        root.geometry("1280x840")


        self.raw = load_raw()
        if self.raw is None:
            raise FileNotFoundError("Не найден файл Bankloan.csv")


        self.df = clean_data(self.raw)


        self.var_n_estimators = tk.IntVar(value=100)
        self.var_lr = tk.StringVar(value="0.1")
        self.var_max_depth = tk.IntVar(value=3)
        self.var_test_size = tk.StringVar(value="0.3")


        self.model = None
        self.best_model = None


        self._build_notebook()
        self._fill_data_tab()
        self._train_model()


    def _build_notebook(self):
        style = ttk.Style(self.root)
        style.configure('Treeview', rowheight=28)


        self.nb = ttk.Notebook(self.root)
        self.nb.pack(fill='both', expand=True)


        self.tab_data = ttk.Frame(self.nb, padding=8)
        self.tab_train = ttk.Frame(self.nb, padding=8)
        self.tab_grid = ttk.Frame(self.nb, padding=8)
        self.tab_influence = ttk.Frame(self.nb, padding=8)


        self.nb.add(self.tab_data, text="Данные и предобработка")
        self.nb.add(self.tab_train, text="Обучение и оценка")
        self.nb.add(self.tab_grid, text="Подбор гиперпараметров")
        self.nb.add(self.tab_influence, text="Влияние параметров")


        self._build_train_tab()
        self._build_grid_tab()
        self._build_influence_tab()


    def _fill_data_tab(self):
        left = ttk.Frame(self.tab_data)
        left.pack(side='left', fill='both', expand=True)


        frm = ttk.LabelFrame(left, text="Исходный датасет Bankloan", padding=6)
        frm.pack(fill='both', expand=True)
```

```
        wrap = ttk.Frame(frm)
        wrap.pack(fill='both', expand=True)
        self.table_data = ttk.Treeview(wrap, show='headings', height=12)
        vsb = ttk.Scrollbar(wrap, orient='vertical',
                            command=self.table_data.yview)
        hsb = ttk.Scrollbar(wrap, orient='horizontal',
                            command=self.table_data.xview)
        self.table_data.configure(yscrollcommand=vsb.set,
                                  xscrollcommand=hsb.set)
        cols = list(self.raw.columns)
        self.table_data.configure(columns=cols)
        for c in cols:
            self.table_data.heading(c, text=c)
            self.table_data.column(c, width=95, anchor='center', stretch=False)
        for _, row in self.raw.head(40).iterrows():
            self.table_data.insert('', 'end', values=[row[c] for c in cols])
        self.table_data.pack(side='left', fill='both', expand=True)
        vsb.pack(side='right', fill='y')
        hsb.pack(side='bottom', fill='x')


        frm = ttk.LabelFrame(left, text="Сводка по предобработке", padding=6)
        frm.pack(fill='x', pady=(8, 0))
        na_target = int(self.raw[TARGET].isna().sum())
        text = (
            f"Исходный размер датасета: {self.raw.shape}\n"
            f"Размер после очистки: {self.df.shape}\n"
            f"Признаков без целевого: {self.df.shape[1] - 1}\n\n"
            "Обработка пропусков: в целевом признаке default пропущено "
            f"{na_target} строк (~{na_target / len(self.raw):.0%}). "
            "Так как метка класса неизвестна, эти строки удалены.\n"
            "Выбросы: в непрерывных признаках значения ограничены "
            "границами IQR (Q1 - 1.5*IQR, Q3 + 1.5*IQR) методом clip().\n"
            "Кодирование: все признаки уже числовые, категориальные "
            "не требуются.\n"
            "Масштабирование: StandardScaler обучен на обучающей выборке "
            "и применён к обучающей и тестовой.\n"
            "Целевой признак: default (1 - дефолт, 0 - нет дефолта).")
        txt = tk.Text(frm, wrap='word', height=12)
        txt.insert('1.0', text)
        txt.configure(state='disabled')
        txt.pack(fill='both', expand=True)


        right = ttk.Frame(self.tab_data)
        right.pack(side='left', fill='both', expand=True, padx=(8, 0))


        frm = ttk.LabelFrame(right, text="Пропущенные значения", padding=6)
        frm.pack(fill='both', expand=True)
        self.fig_missing = plt.Figure(figsize=(6, 3.3))
        fig_to_tab(self.fig_missing, frm)


        frm = ttk.LabelFrame(right, text="Распределение классов", padding=6)
        frm.pack(fill='both', expand=True, pady=(8, 0))
        self.fig_class = plt.Figure(figsize=(6, 3.3))
        fig_to_tab(self.fig_class, frm)


        self._draw_missing()
        self._draw_class()


    def _draw_missing(self):
        ax = self.fig_missing.add_subplot(111)
        sns.heatmap(self.raw.isnull(), cbar=False, cmap='viridis',
                    yticklabels=False, ax=ax)
        ax.set_title("Тепловая карта пропусков (жёлтый — пропуск)")
        self.fig_missing.tight_layout()
        self.fig_missing.canvas.draw_idle()


    def _draw_class(self):
        ax = self.fig_class.add_subplot(111)
        sns.countplot(x=self.df[TARGET], hue=self.df[TARGET], legend=False,
                      ax=ax, palette=['#4c72b0', '#c44e52'])
        ax.set_xticks([0, 1])
        ax.set_xticklabels(['0 (нет дефолта)', '1 (дефолт)'])
        ax.set_xlabel("")
        ax.set_title(f"Классы после удаления пропусков (n={len(self.df)})")
        self.fig_class.tight_layout()
        self.fig_class.canvas.draw_idle()
```

```
@staticmethod
def _field(parent, label, widget):
    row = ttk.Frame(parent)
    row.pack(fill='x', pady=2)
    ttk.Label(row, text=label, width=18).pack(side='left')
    widget.pack(side='left')


@staticmethod
def _parse_float(var, default):
    try:
        return float(var.get().replace(',', '.'))
    except ValueError:
        return default


@staticmethod
def _set_text(widget, text):
    widget.configure(state='normal')
    widget.delete('1.0', 'end')
    widget.insert('1.0', text)
    widget.configure(state='disabled')


@staticmethod
def _setup_metrics_tree(tree):
    cols = ['Модель'] + METRIC_NAMES
    tree.configure(columns=cols)
    for c in cols:
        title = 'Модель' if c == 'Модель' else METRIC_RUS[c]
        tree.heading(c, text=title)
        tree.column(c, width=115, anchor='center')


def _fill_metrics_tree(self, tree, rows):
    for item in tree.get_children():
        tree.delete(item)
    for label, metrics in rows:
        values = [label] + [f"{metrics[m]:.4f}" for m in METRIC_NAMES]
        tree.insert('', 'end', values=values)


def _build_train_tab(self):
    top = ttk.Frame(self.tab_train)
    top.pack(fill='x')


    frm = ttk.LabelFrame(top, text="Параметры градиентного бустинга",
                         padding=6)
    frm.pack(side='left', fill='y')
    self._field(frm, "Число деревьев:",
                ttk.Spinbox(frm, from_=10, to=500,
                            textvariable=self.var_n_estimators, width=7))
    self._field(frm, "Скорость обучения:",
                ttk.Entry(frm, textvariable=self.var_lr, width=9))
    self._field(frm, "Глубина дерева:",
                ttk.Spinbox(frm, from_=1, to=8,
                            textvariable=self.var_max_depth, width=7))


    frm2 = ttk.LabelFrame(top, text="Разбиение выборки", padding=6)
    frm2.pack(side='left', fill='y', padx=8)
    self._field(frm2, "Доля теста:",
                ttk.Entry(frm2, textvariable=self.var_test_size, width=9))
    ttk.Label(frm2, text="стратификация по классам").pack(anchor='w')


    btns = ttk.Frame(top)
    btns.pack(side='left', fill='y')
    ttk.Button(btns, text="Обучить модель",
               command=self._train_model).pack(fill='x')
    ttk.Button(btns, text="Сбросить параметры",
               command=self._reset_params).pack(fill='x', pady=(6, 0))


    mid = ttk.Frame(self.tab_train)
    mid.pack(fill='both', expand=True, pady=(8, 0))


    frm = ttk.LabelFrame(mid, text="Метрики качества на тестовой выборке",
                         padding=6)
    frm.pack(side='left', fill='both', expand=True)
    self.tree_metrics = ttk.Treeview(frm, show='headings', height=6)
    self._setup_metrics_tree(self.tree_metrics)
    self.tree_metrics.pack(fill='both', expand=True)
```

```
        frm = ttk.LabelFrame(mid, text="Отчёт классификации", padding=6)
        frm.pack(side='left', fill='both', expand=True, padx=(8, 0))
        self.txt_train = tk.Text(frm, wrap='word', height=9,
                                 font=("Courier New", 9))
        self.txt_train.pack(fill='both', expand=True)

        bottom = ttk.Frame(self.tab_train)
        bottom.pack(fill='both', expand=True, pady=(8, 0))

        frm = ttk.LabelFrame(bottom, text="Матрица ошибок", padding=6)
        frm.pack(side='left', fill='both', expand=True)
        self.fig_cm = plt.Figure(figsize=(4.8, 3.5))
        fig_to_tab(self.fig_cm, frm)

        frm = ttk.LabelFrame(bottom, text="ROC-кривая", padding=6)
        frm.pack(side='left', fill='both', expand=True, padx=(8, 0))
        self.fig_roc = plt.Figure(figsize=(4.8, 3.5))
        fig_to_tab(self.fig_roc, frm)

    def _build_grid_tab(self):
        top = ttk.Frame(self.tab_grid)
        top.pack(fill='x')
        ttk.Label(top, text="Сетка: n_estimators [50, 100, 150], "
                            "learning_rate [0.05, 0.1, 0.2], max_depth [2, 3]",
                  foreground='#555').pack(side='left')
        ttk.Button(top, text="Запустить GridSearchCV",
                   command=self._run_grid_search).pack(side='left', padx=12)

        mid = ttk.Frame(self.tab_grid)
        mid.pack(fill='both', expand=True, pady=(8, 0))

        frm = ttk.LabelFrame(mid, text="Сравнение метрик", padding=6)
        frm.pack(side='left', fill='both', expand=True)
        self.tree_compare = ttk.Treeview(frm, show='headings', height=6)
        self._setup_metrics_tree(self.tree_compare)
        self.tree_compare.pack(fill='both', expand=True)

        frm = ttk.LabelFrame(mid, text="Оптимальные параметры", padding=6)
        frm.pack(side='left', fill='both', expand=True, padx=(8, 0))
        self.txt_grid = tk.Text(frm, wrap='word', height=9,
                                font=("Courier New", 9))
        self.txt_grid.pack(fill='both', expand=True)

        bottom = ttk.Frame(self.tab_grid)
        bottom.pack(fill='both', expand=True, pady=(8, 0))

        frm = ttk.LabelFrame(bottom, text="Матрица ошибок: исходная модель",
                             padding=6)
        frm.pack(side='left', fill='both', expand=True)
        self.fig_cm_init = plt.Figure(figsize=(5, 3.3))
        fig_to_tab(self.fig_cm_init, frm)

        frm = ttk.LabelFrame(bottom, text="Матрица ошибок: настроенная модель",
                             padding=6)
        frm.pack(side='left', fill='both', expand=True, padx=(8, 0))
        self.fig_cm_best = plt.Figure(figsize=(5, 3.3))
        fig_to_tab(self.fig_cm_best, frm)

    def _build_influence_tab(self):
        top = ttk.Frame(self.tab_influence)
        top.pack(fill='x')
        ttk.Button(top, text="Исследовать влияние learning_rate и числа деревьев",
                   command=self._run_influence).pack(side='left')
        ttk.Label(top, text="Фиксировано: max_depth=3, число деревьев 10…150",
                  foreground='#555').pack(side='left', padx=12)

        frm = ttk.LabelFrame(self.tab_influence,
                             text="Accuracy от числа деревьев при разных "
                                  "learning_rate", padding=6)
        frm.pack(fill='both', expand=True, pady=(8, 0))
        self.fig_influence = plt.Figure(figsize=(11, 4.8))
        fig_to_tab(self.fig_influence, frm)
```

```
    frm = ttk.LabelFrame(self.tab_influence, text="Важность признаков",
                         padding=6)
    frm.pack(fill='both', expand=True, pady=(8, 0))
    self.fig_importance = plt.Figure(figsize=(11, 2.6))
    fig_to_tab(self.fig_importance, frm)


def _prepare(self):
    X = self.df.drop(columns=[TARGET])
    y = self.df[TARGET]
    test_size = self._parse_float(self.var_test_size, 0.3)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y)
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    self.feature_names = list(X.columns)
    self.X_train = X_train
    self.X_test = X_test
    self.y_train = y_train
    self.y_test = y_test
    self.scaler = scaler


def _reset_params(self):
    self.var_n_estimators.set(100)
    self.var_lr.set("0.1")
    self.var_max_depth.set(3)
    self.var_test_size.set("0.3")


def _train_model(self):
    self._prepare()
    lr = self._parse_float(self.var_lr, 0.1)
    self.model = GradientBoostingClassifier(
        n_estimators=int(self.var_n_estimators.get()),
        learning_rate=lr,
        max_depth=int(self.var_max_depth.get()),
        random_state=42)
    self.model.fit(self.X_train, self.y_train)
    self.metrics_initial, self.pred_initial, self.proba_initial = evaluate(
        self.model, self.X_test, self.y_test)
    self._show_initial()


def _show_initial(self):
    self._fill_metrics_tree(self.tree_metrics,
                            [('Исходная модель', self.metrics_initial)])
    report = classification_report(self.y_test, self.pred_initial,
                                   target_names=['0 (нет)', '1 (да)'])
    text = (f"Объектов в обучении: {len(self.y_train)}, "
            f"в тесте: {len(self.y_test)}\n"
            f"Признаки: {', '.join(self.feature_names)}\n"
            f"Модель: GradientBoostingClassifier("
            f"n_estimators={self.model.n_estimators}, "
            f"learning_rate={self.model.learning_rate}, "
            f"max_depth={self.model.max_depth})\n\n"
            + report)
    self._set_text(self.txt_train, text)
    self._draw_confusion(self.fig_cm, self.pred_initial,
                         "Исходная модель")
    self._draw_roc(self.fig_roc,
                   [('Исходная модель', self.proba_initial)])


def _draw_confusion(self, fig, y_pred, title):
    fig.clear()
    ax = fig.add_subplot(111)
    cm = confusion_matrix(self.y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax,
                xticklabels=['0', '1'], yticklabels=['0', '1'])
    ax.set_xlabel("Предсказано")
    ax.set_ylabel("Факт")
    ax.set_title(title)
    fig.tight_layout()
    fig.canvas.draw_idle()


def _draw_roc(self, fig, curves):
    fig.clear()
    ax = fig.add_subplot(111)
    for name, proba in curves:
```

```
            fpr, tpr, _ = roc_curve(self.y_test, proba)
            auc = roc_auc_score(self.y_test, proba)
            ax.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
        ax.plot([0, 1], [0, 1], 'k--', lw=1)
        ax.set_xlabel("False Positive Rate")
        ax.set_ylabel("True Positive Rate")
        ax.set_title("ROC-кривая")
        ax.legend(loc='lower right')
        fig.tight_layout()
        fig.canvas.draw_idle()

    def _run_grid_search(self):
        if self.model is None:
            self._train_model()
        param_grid = {
            'n_estimators': [50, 100, 150],
            'learning_rate': [0.05, 0.1, 0.2],
            'max_depth': [2, 3],
        }
        gs = GridSearchCV(GradientBoostingClassifier(random_state=42),
                          param_grid, cv=3, scoring='f1', n_jobs=-1)
        gs.fit(self.X_train, self.y_train)
        self.best_model = gs.best_estimator_
        self.metrics_best, self.pred_best, self.proba_best = evaluate(
            self.best_model, self.X_test, self.y_test)


        self._fill_metrics_tree(self.tree_compare, [
            ('Исходная модель', self.metrics_initial),
            ('Настроенная модель', self.metrics_best)])


        results = pd.DataFrame(gs.cv_results_)
        results = results.sort_values('rank_test_score').head(5)
        lines = [f"Лучшие параметры: {gs.best_params_}",
                 f"Лучший F1 на кросс-валидации: {gs.best_score_:.4f}",
                 "",
                 "Топ-5 комбинаций по CV:",
                 "n_estimators  learning_rate  max_depth  F1"]
        for _, r in results.iterrows():
            lines.append(f"{int(r['param_n_estimators']):<13}"
                         f"{r['param_learning_rate']:<15}"
                         f"{int(r['param_max_depth']):<10}"
                         f"{r['mean_test_score']:.4f}")
        self._set_text(self.txt_grid, "\n".join(lines))
        self._draw_confusion(self.fig_cm_init, self.pred_initial,
                             "Исходная модель")
        self._draw_confusion(self.fig_cm_best, self.pred_best,
                             "Настроенная модель")


    def _run_influence(self):
        if self.model is None:
            self._train_model()
        n_trees = [10, 20, 50, 100, 150]
        rates = [0.01, 0.05, 0.1, 0.5]
        self.fig_influence.clear()
        for i, lr in enumerate(rates, 1):
            train_acc = []
            test_acc = []
            for n in n_trees:
                m = GradientBoostingClassifier(
                    n_estimators=n, learning_rate=lr, max_depth=3,
                    random_state=42)
                m.fit(self.X_train, self.y_train)
                train_acc.append(
                    accuracy_score(self.y_train, m.predict(self.X_train)))
                test_acc.append(
                    accuracy_score(self.y_test, m.predict(self.X_test)))
            ax = self.fig_influence.add_subplot(2, 2, i)
            ax.plot(n_trees, train_acc, marker='o', label='Обучающая')
            ax.plot(n_trees, test_acc, marker='s', label='Тестовая')
            ax.set_title(f"learning_rate = {lr}")
            ax.set_xlabel("Число деревьев")
            ax.set_ylabel("Accuracy")
            ax.set_xticks(n_trees)
            ax.grid(True)
            ax.legend(fontsize=8)
        self.fig_influence.tight_layout()
        self.fig_influence.canvas.draw_idle()
        self._draw_importance()
```

```
    def _draw_importance(self):
        model = self.best_model if self.best_model is not None else self.model
        importances = model.feature_importances_
        order = np.argsort(importances)
        names = [self.feature_names[i] for i in order]
        values = importances[order]
        self.fig_importance.clear()
        ax = self.fig_importance.add_subplot(111)
        ax.barh(names, values, color='seagreen')
        ax.set_xlabel("Важность")
        ax.set_title("Важность признаков в градиентном бустинге")
        self.fig_importance.tight_layout()
        self.fig_importance.canvas.draw_idle()


def main():
    root = tk.Tk()
    Lab6App(root)
    root.mainloop()


if __name__ == '__main__':
    main()
```

## 5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

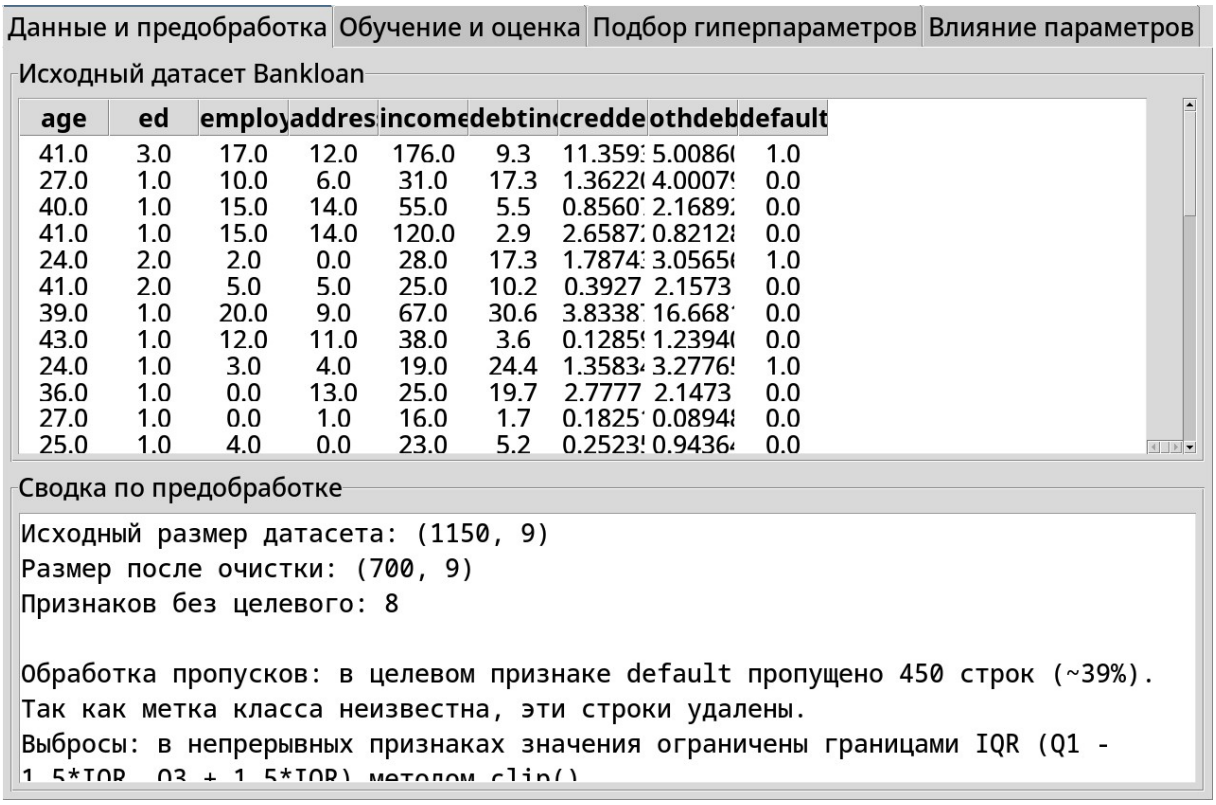



*Рисунок 1 — Вкладка «Данные и предобработка»: таблица датасета, тепловая карта пропусков, распределение классов и сводка обработки*

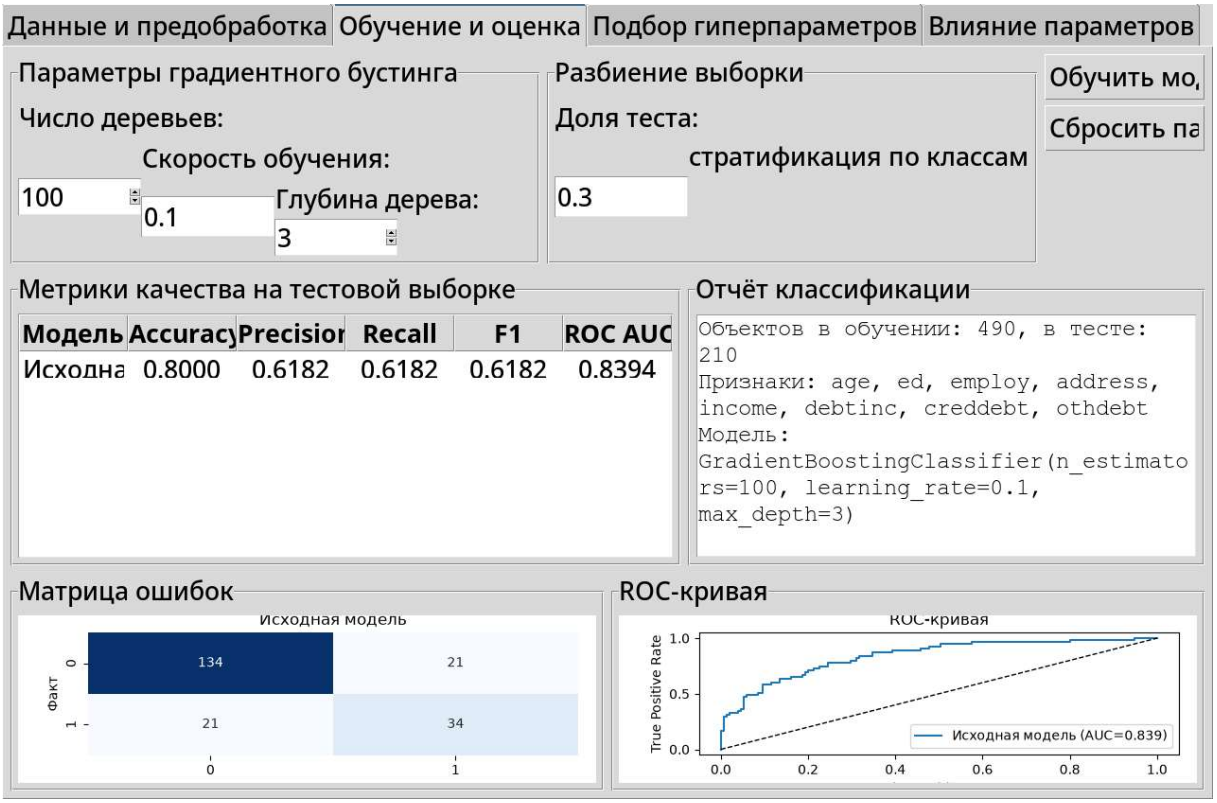



*Рисунок 2 — Вкладка «Обучение и оценка»: параметры модели, метрики, отчёт классификации, матрица ошибок и ROC-кривая*

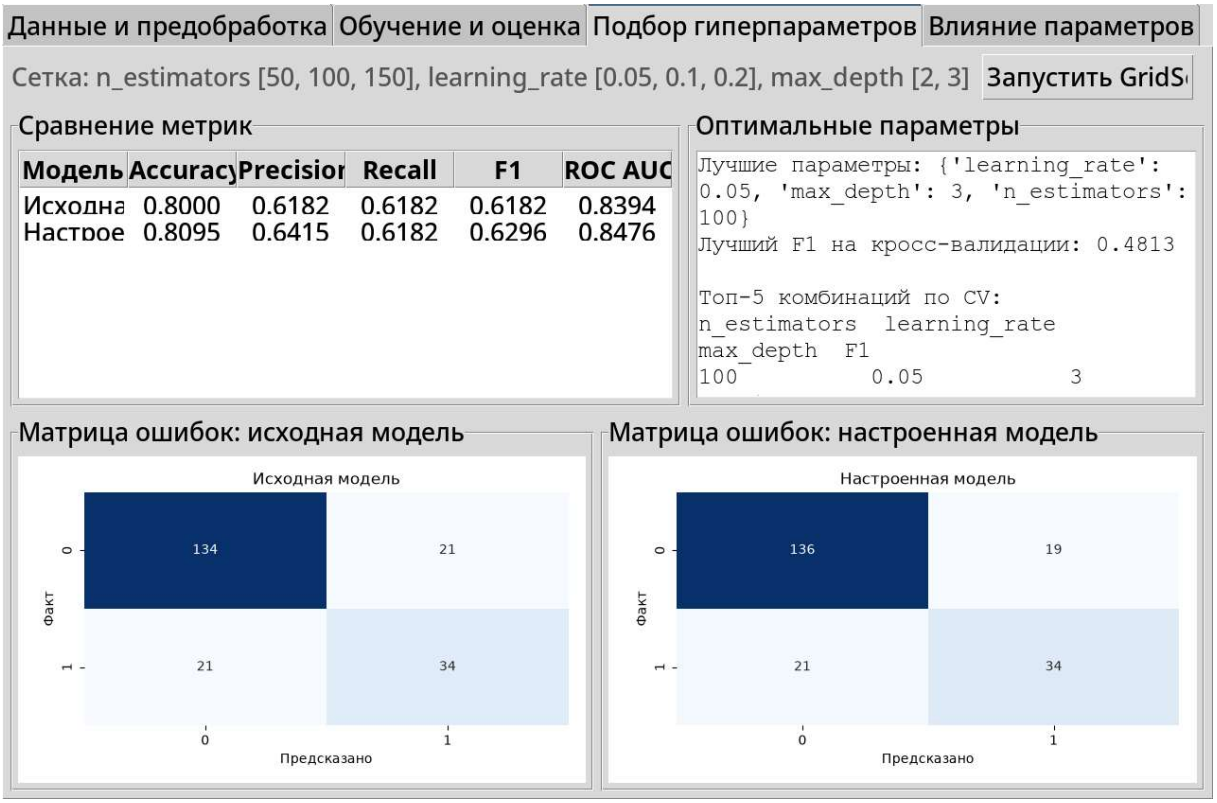



*Рисунок 3 — Вкладка «Подбор гиперпараметров»: результаты GridSearchCV и сравнение матриц ошибок исходной и настроенной модели*

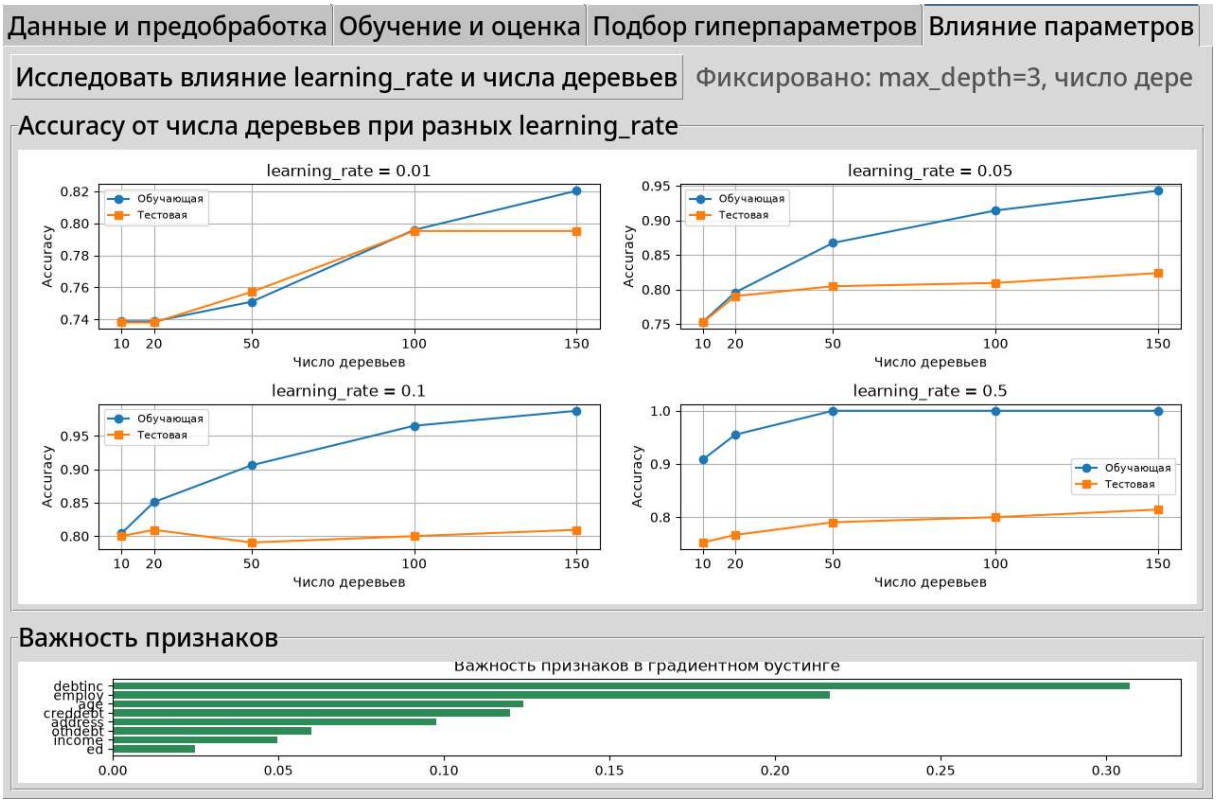



*Рисунок 4 — Вкладка «Влияние параметров»: зависимость accuracy от числа деревьев при разных learning_rate и важность признаков*

## 6 ВЫВОДЫ ПО ЛАБОРАТОРНОЙ РАБОТЕ

В ходе выполнения лабораторной работы был изучен и практически применён ансамблевый метод градиентного бустинга для задачи бинарной классификации. Использован датасет Bankloan, содержащий 1150 наблюдений и 8 признаков. В целевом признаке default отсутствовало 450 значений (~39% строк); так как метка класса неизвестна, эти строки удалены, после чего осталось 700 наблюдений. Выбросы в непрерывных признаках были ограничены границами IQR методом clip(), признаки стандартизированы с помощью StandardScaler (обучен только на обучающей выборке). Данные разбиты на обучающую и тестовую части в пропорции 70/30 со стратификацией по классам.

Исходная модель GradientBoostingClassifier (n_estimators=100, learning_rate=0.1, max_depth=3) показала на тестовой выборке accuracy = 0.800, precision = 0.618, recall = 0.618, F1 = 0.618 и ROC AUC = 0.839. Построены матрица ошибок и ROC-кривая, а также отчёт классификации.

С помощью GridSearchCV (сетка n_estimators {50, 100, 150}, learning_rate {0.05, 0.1, 0.2}, max_depth {2, 3}, cv=3, критерий f1) найдены оптимальные параметры: n_estimators=100, learning_rate=0.05, max_depth=3. После повторного обучения

метрики улучшились: accuracy = 0.810, precision = 0.642, recall = 0.618, F1 = 0.630, ROC AUC = 0.848, что подтверждает пользу подбора гиперпараметров.

Исследование влияния скорости обучения и числа деревьев показало: при малой скорости обучения (0.01) точность растёт медленно и требует большего числа деревьев; при слишком большой скорости (0.5) обучающая точность достигает 1.0, что свидетельствует о переобучении, тогда как тестовая точность остаётся на уровне 0.79–0.81. Наиболее сбалансированными оказались значения learning_rate 0.05–0.1. С ростом числа деревьев качество повышается, однако после 100–150 деревьев рост замедляется.

Анализ важности признаков показал, что наибольший вклад в предсказание дефолта вносят признаки debtinc (0.279), employ (0.182), creddebt (0.143) и age (0.120); наименьший — ed (0.024). Полученные результаты согласуются с содержательным смыслом задачи кредитного скоринга.